

Mission 004 Checkpoint

Generated: 2026-05-27T16:46:23Z

Mission

Mission 004: Live Truth Ingestion and Storytelling

Current state

Mission 004 is complete. The canonical live JSON source was refreshed again in the repo with a new Mission 004 verification event, production ingests the refreshed feed, and the deployed app shows Mission 004 with freshness visible instead of hidden.

Current production smoke from this run:

- `GET https://agents-vis.vercel.app/api/missions/latest` → Mission 004, `eventCount: 7`, `freshnessState: delayed`, `lagMs: 586166`
- `GET https://agents-vis.vercel.app/api/dashboard` → Mission 004, delayed freshness still visible, `lagMs: 588161`
- Browser title: `Mission 004`

Current local verification from this run:

- `AGENTS\_VIS\_DASHBOARD\_SOURCE\_FILE=src/lib/dashboard-live-source.json`
- `npm test` passed with the route test pinned to the repo-backed live source file
- `npm run typecheck` passed
- `npm run build` passed

Latest known mission scope

- Keep the single latest-mission timeline experience
- Make the backend production-safe and source-driven
- Keep the displayed truth current through a real ingestion path
- Preserve chronology, parallel metadata, and freshness states
- Keep the app read-only and consumer-friendly
- Improve the story quality of what agents write

What is done

- Mission 004 scope has been defined
- Product, backend, frontend, QA, and coordinator responsibilities are written down
- The Mission 004 writing contract is defined: updates should communicate what changed, who changed it, why it matters, and what comes next when relevant
- The canonical live JSON source was refreshed with Mission 004 as the latest mission and a clearer story update, then refreshed again with a new verification event
- The backend can poll `AGENTS\_VIS\_DASHBOARD\_SOURCE\_URL` and also falls back to the verified raw GitHub JSON feed
- Tests, typecheck, and build pass in the repo
- Local verification from this run is green: `npm test`, `npm run typecheck`, and `npm run build` all passed
- Mission 004 PDFs were regenerated locally with ReportLab so the repo-hosted artifacts match the updated markdown
- The refreshed live source was committed and pushed to `main`
- Live production smoke confirms `https://agents-vis.vercel.app/api/missions/latest` returns Mission 004 data with 7 events, `https://agents-vis.vercel.app/api/dashboard` returns Mission 004, and the page title says Mission 004
- Production now returns Mission 004 from the refreshed canonical live source with delayed freshness visible
- The route test now pins to the repo-backed live source file so local verification tracks the shipped source instead of the remote feed
- The latest production smoke in this run returned Mission 004 with visible delayed freshness and `lagMs` in the live range

What remains

- No functional blocker remains for Mission 004

- Future runs should keep the canonical live source refreshed as new truth arrives
- Keep the one-timeline experience, the read-only API, and visible freshness states intact

Resume instruction

If this mission is reopened, read the checkpoint, mission packet, and handoffs, then refresh the live JSON source before making any UI or documentation changes.

Latest run summary

- Coordinator verified repo quality gates, refreshed the live source, pushed the commit, and re-smoked production
- The repo-backed live source now includes Mission 004 as the latest mission with clearer story text
- Backend/frontend code can poll `AGENTS\_VIS\_DASHBOARD\_SOURCE\_URL`, and this run verified the live GitHub JSON feed locally against the app
- QA evidence now shows the deployed app on Mission 004 with delayed freshness visible
- Repo-hosted PDFs were regenerated to match the updated markdown

Notes for the next run

- Do not add a second dashboard view
- Do not hide lag or stale states
- Preserve parallel sequencing
- Keep status tied to live evidence and production checks
- Production smoke currently reaches the deployed app, and the live payload is Mission 004 with delayed freshness visible
- Send short updates after every meaningful step